

EduRAG

AI-Powered Education Platform

Complete Technical Specification for AI Agent–Based Development

Backend	C# / ASP.NET Core 8
Frontend	React 18 + TypeScript
Database	PostgreSQL 16 + pgvector
AI Engine	Ollama (llama3.2 + nomic-embed-text)
Architecture	Clean Architecture (DDD)
Auth	JWT — Admin & Student roles

Version 1.0 • Complete AI Agent Build Guide

1. Project Overview

EduRAG is a full-stack, AI-powered education platform that enables admins to manage curriculum content and allows students to learn interactively using RAG (Retrieval-Augmented Generation) powered chat. The system runs entirely on free, self-hosted AI — no paid APIs, no token costs.

KEY DESIGN PRINCIPLE: All AI runs locally via Ollama. No OpenAI, Anthropic, or any paid API is used. The embedding model is nomic-embed-text and the chat model is llama3.2. Both run on the same server as the application.

1.1 Core Features

Admin Capabilities

- Create and manage Classes (Grade 1 to 12)
- Create Subjects under each Class
- Create Chapters under each Subject
- Upload PDF study materials per Class / Subject / Chapter
- View vectorization status of uploaded materials
- Manage admin and student user accounts

Student Capabilities

- Login and select a Class and Subject
- Open a chat window scoped to that Class + Subject
- Ask free-form questions — answers are generated from uploaded PDFs
- Request practice questions for any topic or chapter
- Submit answers to practice questions and receive graded feedback
- View explanation of correct answers when incorrect

1.2 Technology Stack

Component	Technology	Version / Notes
Backend API	ASP.NET Core Web API	C# 12, .NET 8
Frontend	React + TypeScript	React 18, Vite, Tailwind CSS v3
Database	PostgreSQL	v16 with pgvector extension
Vector Search	pgvector	Cosine similarity, HNSW index
ORM (Writes)	Entity Framework Core	EF Core 8 — Insert/Update/Delete only

Component	Technology	Version / Notes
Micro-ORM (Reads)	Dapper	v2.1 — All SELECT queries
AI Runtime	Ollama	v0.3+ — local HTTP server
Chat LLM	llama3.2	8B param — free, local
Embedding Model	nomic-embed-text	768-dim vectors — free, local
PDF Processing	PdfPig	C# MIT library
Auth	JWT Bearer Tokens	Role-based: Admin / Student
Background Jobs	IHostedService + Channel<T>	Built-in .NET — no extra infra
Containerization	Docker + docker-compose	PostgreSQL + Ollama + API

2. Clean Architecture Structure

The backend follows Clean Architecture (also called Onion Architecture). Dependencies point inward only: API → Application → Domain. Infrastructure implements interfaces defined in Application.

2.1 Solution Layout



2.2 Dependency Rules

RULE: The Domain layer has ZERO NuGet dependencies. The Application layer depends only on Domain. Infrastructure implements Application interfaces. The API project wires everything together via DI.

Project	Allowed References	Forbidden References
EduRAG.Domain	None	Everything
EduRAG.Application	EduRAG.Domain	Infrastructure, API
EduRAG.Infrastructure	EduRAG.Domain, EduRAG.Application	EduRAG.API
EduRAG.API	All three above	None

3. Domain Layer — Complete Entity Models

All entities live in `EduRAG.Domain/Entities`. They are pure C# POCOs with no EF or framework attributes.

3.1 Class.cs

```
namespace EduRAG.Domain.Entities;

public class Class
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty; // e.g. "Class 6"
    public int Grade { get; set; } // 1-12
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public ICollection<Subject> Subjects { get; set; } = new List<Subject>();
}
```

3.2 Subject.cs

```
public class Subject
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public int ClassId { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public Class Class { get; set; } = null!;
    public ICollection<Chapter> Chapters { get; set; } = new List<Chapter>();
    public ICollection<StudyMaterial> Materials { get; set; } = new
List<StudyMaterial>();
}
```

3.3 Chapter.cs

```
public class Chapter
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public int OrderIndex { get; set; }
    public int SubjectId { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public Subject Subject { get; set; } = null!;
    public ICollection<StudyMaterial> Materials { get; set; } = new
List<StudyMaterial>();
}
```

3.4 StudyMaterial.cs

```

public class StudyMaterial
{
    public Guid      Id                { get; set; } = Guid.NewGuid();
    public string    OriginalFileName { get; set; } = string.Empty;
    public string    StoredFilePath   { get; set; } = string.Empty;
    public string    ContentHash      { get; set; } = string.Empty; // SHA-256
    public long      FileSizeBytes    { get; set; }
    public int       ClassId          { get; set; }
    public int       SubjectId        { get; set; }
    public int?      ChapterId        { get; set; }
    public Guid      UploadedById     { get; set; }
    public DateTime  UploadedAt       { get; set; } = DateTime.UtcNow;
    public VectorizationStatus VectorizationStatus { get; set; } =
VectorizationStatus.Pending;
    public string?   VectorizationError { get; set; }

    public Subject   Subject   { get; set; } = null!;
    public Chapter?  Chapter   { get; set; }
    public ICollection<MaterialChunk> Chunks { get; set; } = new List<MaterialChunk>();
}

```

3.5 MaterialChunk.cs — Vector Store Row

```

/// <summary>
/// One text chunk extracted from a PDF, plus its 768-dim nomic-embed-text vector.
/// Stored in PostgreSQL with pgvector. The Embedding column type is vector(768).
/// </summary>
public class MaterialChunk
{
    public Guid      Id                { get; set; } = Guid.NewGuid();
    public Guid      MaterialId        { get; set; }
    public int       ClassId           { get; set; } // denormalized — enables fast filtered
search
    public int       SubjectId         { get; set; } // denormalized
    public int?      ChapterId         { get; set; } // denormalized
    public string    Content           { get; set; } = string.Empty;
    public int       ChunkIndex        { get; set; }
    public int       PageNumber        { get; set; }
    public float[]   Embedding         { get; set; } = Array.Empty<float>(); // mapped to
vector(768)

    public StudyMaterial Material { get; set; } = null!;
}

```

3.6 AppUser.cs, ChatSession.cs, ChatMessage.cs

```

public class AppUser
{
    public Guid      Id                { get; set; } = Guid.NewGuid();
    public string    Email             { get; set; } = string.Empty;
    public string    FullName          { get; set; } = string.Empty;
    public string    PasswordHash      { get; set; } = string.Empty; // BCrypt hashed
    public UserRole   Role              { get; set; } // Admin | Student
    public bool      IsActive          { get; set; } = true;
    public DateTime  CreatedAt         { get; set; } = DateTime.UtcNow;
    public DateTime? LastLoginAt       { get; set; }
    public ICollection<ChatSession> ChatSessions { get; set; } = new
List<ChatSession>();
}

public class ChatSession

```

```
{
    public Guid      Id          { get; set; } = Guid.NewGuid();
    public Guid      UserId      { get; set; }
    public int       ClassId     { get; set; }
    public int       SubjectId   { get; set; }
    public DateTime   StartedAt  { get; set; } = DateTime.UtcNow;
    public DateTime? EndedAt     { get; set; }
    public AppUser   User        { get; set; } = null!;
    public ICollection<ChatMessage> Messages { get; set; } = new List<ChatMessage>();
}

public class ChatMessage
{
    public Guid      Id          { get; set; } = Guid.NewGuid();
    public Guid      SessionId   { get; set; }
    public string     Content     { get; set; } = string.Empty;
    public MessageRole Role      { get; set; } // User | Assistant
    public DateTime   SentAt     { get; set; } = DateTime.UtcNow;
    public string?    SourceChunkIds { get; set; } // JSON array of chunk IDs used
    public ChatSession Session    { get; set; } = null!;
}
```

3.7 Enums

```
namespace EduRAG.Domain.Enums;

public enum UserRole          { Admin, Student }
public enum MessageRole      { User, Assistant }
public enum VectorizationStatus { Pending, Processing, Completed, Failed }
```


4. Database Schema — PostgreSQL + pgvector

4.1 Initial Setup

```
-- Run once on the PostgreSQL server
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- for gen_random_uuid()
```

4.2 Full Schema SQL

```
CREATE TABLE "Classes" (
  "Id"          SERIAL PRIMARY KEY,
  "Name"        VARCHAR(100) NOT NULL,
  "Grade"       INT NOT NULL CHECK ("Grade" BETWEEN 1 AND 12),
  "IsActive"    BOOLEAN NOT NULL DEFAULT TRUE,
  "CreatedAt"   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE "Subjects" (
  "Id"          SERIAL PRIMARY KEY,
  "Name"        VARCHAR(150) NOT NULL,
  "Description"  TEXT NOT NULL DEFAULT '',
  "ClassId"     INT NOT NULL REFERENCES "Classes"("Id") ON DELETE CASCADE,
  "IsActive"    BOOLEAN NOT NULL DEFAULT TRUE,
  "CreatedAt"   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE "Chapters" (
  "Id"          SERIAL PRIMARY KEY,
  "Title"       VARCHAR(200) NOT NULL,
  "OrderIndex"  INT NOT NULL DEFAULT 0,
  "SubjectId"   INT NOT NULL REFERENCES "Subjects"("Id") ON DELETE CASCADE,
  "IsActive"    BOOLEAN NOT NULL DEFAULT TRUE,
  "CreatedAt"   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE "AppUsers" (
  "Id"          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "Email"       VARCHAR(255) NOT NULL UNIQUE,
  "FullName"    VARCHAR(200) NOT NULL,
  "PasswordHash" TEXT NOT NULL,
  "Role"        INT NOT NULL, -- 0=Admin, 1=Student
  "IsActive"    BOOLEAN NOT NULL DEFAULT TRUE,
  "CreatedAt"   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  "LastLoginAt" TIMESTAMPTZ NULL
);

CREATE TABLE "StudyMaterials" (
  "Id"          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "OriginalFileName" VARCHAR(500) NOT NULL,
  "StoredFilePath"  TEXT NOT NULL,
  "ContentHash"     CHAR(64) NOT NULL,
  "FileSizeBytes"   BIGINT NOT NULL DEFAULT 0,
  "ClassId"        INT NOT NULL REFERENCES "Classes"("Id"),
  "SubjectId"       INT NOT NULL REFERENCES "Subjects"("Id"),
  "ChapterId"       INT NOT NULL REFERENCES "Chapters"("Id"),
  "UploadedById"    UUID NOT NULL REFERENCES "AppUsers"("Id"),
  "UploadedAt"      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

        "VectorizationStatus" INT NOT NULL DEFAULT 0, --
        0=Pending,1=Processing,2=Completed,3=Failed
        "VectorizationError" TEXT NULL
    );

CREATE TABLE "MaterialChunks" (
    "Id"          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    "MaterialId"  UUID NOT NULL REFERENCES "StudyMaterials"("Id") ON DELETE CASCADE,
    "ClassId"     INT NOT NULL,
    "SubjectId"   INT NOT NULL,
    "ChapterId"   INT NULL,
    "Content"     TEXT NOT NULL,
    "ChunkIndex"  INT NOT NULL,
    "PageNumber"  INT NOT NULL DEFAULT 1,
    "Embedding"   vector(768) NOT NULL -- nomic-embed-text produces 768-dim vectors
);

-- HNSW index for fast approximate nearest-neighbour search
CREATE INDEX idx_chunks_embedding
    ON "MaterialChunks" USING hnsw ("Embedding" vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Composite index for filtered search (class + subject + vector)
CREATE INDEX idx_chunks_class_subject ON "MaterialChunks" ("ClassId", "SubjectId");

CREATE TABLE "ChatSessions" (
    "Id"          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    "UserId"      UUID NOT NULL REFERENCES "AppUsers"("Id"),
    "ClassId"     INT NOT NULL REFERENCES "Classes"("Id"),
    "SubjectId"   INT NOT NULL REFERENCES "Subjects"("Id"),
    "StartedAt"   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "EndedAt"     TIMESTAMPTZ NULL
);

CREATE TABLE "ChatMessages" (
    "Id"          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    "SessionId"   UUID NOT NULL REFERENCES "ChatSessions"("Id") ON DELETE CASCADE,
    "Content"     TEXT NOT NULL,
    "Role"        INT NOT NULL, -- 0=User, 1=Assistant
    "SentAt"      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "SourceChunkIds" TEXT NULL -- JSON array of chunk UUIDs used as RAG context
);

-- Seed: default admin user (password: Admin@123 - BCrypt hashed)
INSERT INTO "AppUsers" ("Email","FullName","PasswordHash","Role")
VALUES ('admin@edurag.com','System Admin',
'$2a$11$...<bcrypt_hash_here>...',0);

```

5. Infrastructure Layer — EF Core + Dapper

5.1 AppDbContext

```
// EduRAG.Infrastructure/Persistence/AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using EduRAG.Domain.Entities;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) {}

    public DbSet<Class> Classes => Set<Class>();
    public DbSet<Subject> Subjects => Set<Subject>();
    public DbSet<Chapter> Chapters => Set<Chapter>();
    public DbSet<StudyMaterial> StudyMaterials => Set<StudyMaterial>();
    public DbSet<MaterialChunk> MaterialChunks => Set<MaterialChunk>();
    public DbSet<AppUser> AppUsers => Set<AppUser>();
    public DbSet<ChatSession> ChatSessions => Set<ChatSession>();
    public DbSet<ChatMessage> ChatMessages => Set<ChatMessage>();

    protected override void OnModelCreating(ModelBuilder mb)
    {
        mb.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
        mb.HasPostgresExtension("vector");
    }
}
```

5.2 MaterialChunkConfiguration — pgvector mapping

```
// EduRAG.Infrastructure/Persistence/Configurations/MaterialChunkConfiguration.cs
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Pgvector.EntityFrameworkCore;

public class MaterialChunkConfiguration : IEntityTypeConfiguration<MaterialChunk>
{
    public void Configure(EntityTypeBuilder<MaterialChunk> b)
    {
        b.ToTable("MaterialChunks");
        b.HasKey(x => x.Id);

        // Map float[] to pgvector type: vector(768)
        b.Property(x => x.Embedding)
            .HasColumnType("vector(768)");

        b.HasIndex(x => new { x.ClassId, x.SubjectId });

        b.HasOne(x => x.Material)
            .WithMany(m => m.Chunks)
            .HasForeignKey(x => x.MaterialId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}
```

5.3 EF Core Repositories — Write Operations Only

// All repositories follow this pattern: Insert/Update/Delete via EF Core only

```
public interface IClassRepository
{
    Task<Class> CreateAsync(Class entity);
    Task<Class> UpdateAsync(Class entity);
    Task DeleteAsync(int id);
}

public class ClassRepository : IClassRepository
{
    private readonly AppDbContext _ctx;
    public ClassRepository(AppDbContext ctx) => _ctx = ctx;

    public async Task<Class> CreateAsync(Class entity)
    {
        _ctx.Classes.Add(entity);
        await _ctx.SaveChangesAsync();
        return entity;
    }

    public async Task<Class> UpdateAsync(Class entity)
    {
        _ctx.Classes.Update(entity);
        await _ctx.SaveChangesAsync();
        return entity;
    }

    public async Task DeleteAsync(int id)
    {
        var entity = await _ctx.Classes.FindAsync(id)
            ?? throw new KeyNotFoundException($"Class {id} not found");
        _ctx.Classes.Remove(entity);
        await _ctx.SaveChangesAsync();
    }
}
```

5.4 Dapper Queries — Read Operations Only

```
// EduRAG.Infrastructure/Persistence/Queries/ClassQueries.cs
// All SELECT queries use Dapper – fast, raw SQL, no change tracking overhead

public class ClassQueries
{
    private readonly IDbConnection _db;
    public ClassQueries(IDbConnection db) => _db = db;

    public async Task<IEnumerable<ClassWithSubjectsDto>> GetAllWithSubjectsAsync()
    {
        const string sql = @"
            SELECT c.""Id"", c.""Name"", c.""Grade"",
                   s.""Id"" AS SubjectId, s.""Name"" AS SubjectName
            FROM ""Classes"" c
            LEFT JOIN ""Subjects"" s ON s.""ClassId"" = c.""Id""
            WHERE c.""IsActive"" = TRUE
            ORDER BY c.""Grade"", s.""Name"";

        var lookup = new Dictionary<int, ClassWithSubjectsDto>();
        await _db.QueryAsync<ClassWithSubjectsDto, SubjectDto, ClassWithSubjectsDto>(
            sql,
```

```
        (cls, sub) => {
            if (!lookup.TryGetValue(cls.Id, out var c)) { c = cls; lookup[c.Id] = c;
        }

            if (sub != null) c.Subjects.Add(sub);
            return c;
        },
        splitOn: "SubjectId"
    );
    return lookup.Values;
}

public async Task<ClassWithSubjectsDto?> GetByIdAsync(int id)
{
    const string sql = @"
        SELECT c.""Id"", c.""Name"", c.""Grade"",
            s.""Id"" AS SubjectId, s.""Name"" AS SubjectName,
            ch.""Id"" AS ChapterId, ch.""Title"" AS ChapterTitle,
            ch.""OrderIndex""
        FROM ""Classes"" c
        LEFT JOIN ""Subjects"" s ON s.""ClassId"" = c.""Id""
        LEFT JOIN ""Chapters"" ch ON ch.""SubjectId"" = s.""Id""
        WHERE c.""Id"" = @id AND c.""IsActive"" = TRUE
        ORDER BY s.""Name"", ch.""OrderIndex""";
    // ... multi-map as above
}
}
```

6. Application Layer — Interfaces (Ports)

All interfaces below live in EduRAG.Application/Interfaces. Infrastructure provides the implementations. The Application layer never imports Infrastructure.

6.1 AI Interfaces

```
// IAIService.cs - all AI operations
public interface IAIService
{
    /// <summary>Embed text using nomic-embed-text via Ollama. Returns 768-dim
    vector.</summary>
    Task<float[]> GetEmbeddingAsync(string text);

    /// <summary>Stream a RAG chat completion from llama3.2 via Ollama.</summary>
    IAsyncEnumerable<string> StreamChatAsync(
        string systemPrompt,
        IEnumerable<ChatMessageDto> history,
        string userMessage);
}
```

6.2 Vector Search Interface

```
// IVectorSearchService.cs
public interface IVectorSearchService
{
    /// <summary>
    /// Find the top-k most semantically similar chunks for a query embedding.
    /// Results are scoped to classId + subjectId for relevance.
    /// </summary>
    Task<IEnumerable<ChunkSearchResult>> SearchAsync(
        float[] queryEmbedding,
        int classId,
        int subjectId,
        int topK = 5);
}

public record ChunkSearchResult(Guid ChunkId, string Content, double Score, int
    PageNumber);
```

6.3 File Storage Interface

```
// IFileStorageService.cs
public interface IFileStorageService
{
    /// <summary>
    /// Save file to: /storage/materials/{classId}/{subjectId}/{chapterId or 'general'}/
    /// Returns the relative stored path.
    /// </summary>
    Task<string> SaveAsync(Stream fileStream, string fileName, int classId,
        int subjectId, int? chapterId);

    Task DeleteAsync(string storedPath);

    Stream OpenRead(string storedPath);
}
```

6.4 Repository Interfaces

```
// All write repository interfaces follow this pattern

public interface IClassRepository
{
    Task<Class>    CreateAsync(Class entity);
    Task<Class>    UpdateAsync(Class entity);
    Task          DeleteAsync(int id);
}

public interface ISubjectRepository
{
    Task<Subject> CreateAsync(Subject entity);
    Task<Subject> UpdateAsync(Subject entity);
    Task          DeleteAsync(int id);
}

public interface IChapterRepository
{
    Task<Chapter> CreateAsync(Chapter entity);
    Task<Chapter> UpdateAsync(Chapter entity);
    Task          DeleteAsync(int id);
}

public interface IStudyMaterialRepository
{
    Task<StudyMaterial>    CreateAsync(StudyMaterial entity);
    Task                  UpdateStatusAsync(Guid id, VectorizationStatus status,
                                             string? error = null);
    Task                  DeleteAsync(Guid id);
    Task<StudyMaterial?> GetByHashAsync(string contentHash);
}

public interface IMaterialChunkRepository
{
    Task BulkInsertAsync(IEnumerable<MaterialChunk> chunks);
    Task DeleteByMaterialAsync(Guid materialId);
}

public interface IChatRepository
{
    Task<ChatSession> CreateSessionAsync(ChatSession session);
    Task<ChatMessage> AddMessageAsync(ChatMessage message);
}
```

7. AI Layer — Ollama Integration (100% Free)

Ollama runs as a local HTTP server at `http://localhost:11434`. Both models are pulled once and run indefinitely at zero cost.

FREE AI MODELS: nomic-embed-text for embeddings (run: `ollama pull nomic-embed-text`) and llama3.2 for chat (run: `ollama pull llama3.2`). Both are open-source, run locally, and cost nothing.

7.1 OllamaAIService.cs — Complete Implementation

```
// EduRAG.Infrastructure/Services/AI/OllamaAIService.cs
using System.Net.Http.Json;
using System.Runtime.CompilerServices;
using System.Text.Json;

public class OllamaAIService : IAIService
{
    private readonly HttpClient _http;
    private const string EmbedModel = "nomic-embed-text";
    private const string ChatModel = "llama3.2";

    public OllamaAIService(HttpClient http) => _http = http;

    // — EMBEDDINGS —
    public async Task<float[]> GetEmbeddingAsync(string text)
    {
        var response = await _http.PostAsJsonAsync("/api/embeddings", new
        {
            model = EmbedModel,
            prompt = text
        });
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<EmbedResponse>();
        return result!.Embedding;
    }

    private record EmbedResponse([property: JsonPropertyName("embedding")] float[] Embedding);

    // — STREAMING CHAT —
    public async IAsyncEnumerable<string> StreamChatAsync(
        string systemPrompt,
        IEnumerable<ChatMessageDto> history,
        string userMessage,
        [EnumeratorCancellation] CancellationToken ct = default)
    {
        var messages = new List<object>
        {
            new { role = "system", content = systemPrompt }
        };
        foreach (var m in history)
            messages.Add(new { role = m.Role == MessageRole.User ? "user" : "assistant",
                               content = m.Content });
        messages.Add(new { role = "user", content = userMessage });

        var request = new HttpRequestMessage(HttpMethod.Post, "/api/chat")
        {

```



```

        Content = JsonConvert.Create(new { model = ChatModel, messages, stream =
true })
    };

    using var response = await _http.SendAsync(request,
        HttpCompletionOption.ResponseHeadersRead, ct);
    response.EnsureSuccessStatusCode();

    using var stream = await response.Content.ReadAsStreamAsync(ct);
    using var reader = new StreamReader(stream);

    while (!reader.EndOfStream && !ct.IsCancellationRequested)
    {
        var line = await reader.ReadLineAsync();
        if (string.IsNullOrEmpty(line)) continue;

        var chunk = JsonSerializer.Deserialize<ChatStreamChunk>(line);
        if (chunk?.Message?.Content is not null)
            yield return chunk.Message.Content;

        if (chunk?.Done == true) break;
    }
}

private record ChatStreamChunk(
    [property: JsonPropertyName("message")] ChatStreamMessage? Message,
    [property: JsonPropertyName("done")] bool Done);
private record ChatStreamMessage(
    [property: JsonPropertyName("content")] string? Content);
}

```

7.2 VectorSearchService.cs — pgvector cosine search

```

// EduRAG.Infrastructure/Services/VectorSearchService.cs
// Uses Dapper + pgvector to run fast approximate nearest-neighbour queries

public class VectorSearchService : IVectorSearchService
{
    private readonly IDbConnection _db;
    public VectorSearchService(IDbConnection db) => _db = db;

    public async Task<IEnumerable<ChunkSearchResult>> SearchAsync(
        float[] queryEmbedding, int classId, int subjectId, int topK = 5)
    {
        // Format float[] as PostgreSQL vector literal: '[0.1,0.2,...]'
        var vectorLiteral = '[' + string.Join(',', queryEmbedding) + ']';

        const string sql = @"
            SELECT
                ""Id"" AS ChunkId,
                ""Content"" AS Content,
                ""PageNumber"" AS PageNumber,
                1 - (""Embedding"" <=> @vector::vector) AS Score
            FROM ""MaterialChunks""
            WHERE ""ClassId"" = @classId
              AND ""SubjectId"" = @subjectId
            ORDER BY ""Embedding"" <=> @vector::vector
            LIMIT @topK";

        return await _db.QueryAsync<ChunkSearchResult>(sql,
            new { vector = vectorLiteral, classId, subjectId, topK });
    }
}

```

```
}
}
```

7.3 PdfProcessingService.cs — Text Extraction + Chunking

```
// EduRAG.Infrastructure/Services/AI/PdfProcessingService.cs
// Uses PdfPig (MIT) – no cost, no API calls

public class PdfProcessingService
{
    private const int ChunkSize    = 500; // target tokens per chunk (approx chars/4)
    private const int ChunkOverlap = 50;  // overlap for context continuity

    public List<(string Text, int Page)> ExtractAndChunk(Stream pdfStream)
    {
        var results = new List<(string, int)>();

        using var doc = UglyToad.PdfPig.PdfDocument.Open(pdfStream);
        var fullPageTexts = new List<(string Text, int Page)>();

        foreach (var page in doc.GetPages())
        {
            var text = string.Join(' ', page.GetWords().Select(w => w.Text));
            if (!string.IsNullOrEmpty(text))
                fullPageTexts.Add((text.Trim(), page.Number));
        }

        // Slide a window across each page's text
        foreach (var (pageText, pageNum) in fullPageTexts)
        {
            var words = pageText.Split(' ', StringSplitOptions.RemoveEmptyEntries);
            var i = 0;
            while (i < words.Length)
            {
                var chunk = string.Join(' ', words.Skip(i).Take(ChunkSize));
                results.Add((chunk, pageNum));
                i += ChunkSize - ChunkOverlap;
            }
        }
        return results;
    }
}
```

7.4 VectorizationWorker.cs — Background Job

```
// EduRAG.Infrastructure/BackgroundJobs/VectorizationWorker.cs
// Reads from an in-memory Channel<Guid> – no queue infrastructure needed

public class VectorizationWorker : BackgroundService
{
    private readonly Channel<Guid> _queue;
    private readonly IServiceScopeFactory _scopeFactory;

    public VectorizationWorker(Channel<Guid> queue, IServiceScopeFactory sf)
    { _queue = queue; _scopeFactory = sf; }

    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        await foreach (var materialId in _queue.Reader.ReadAllAsync(ct))
        {

```

```
        using var scope = _scopeFactory.CreateScope();
        var processor =
scope.ServiceProvider.GetRequiredService<VectorizationProcessor>();
        await processor.ProcessAsync(materialId, ct);
    }
}

// VectorizationProcessor.cs – the actual embedding work
public class VectorizationProcessor
{
    // Dependencies injected: IStudyMaterialRepository, IMaterialChunkRepository,
    // IFileStorageService, PdfProcessingService, IAIService

    public async Task ProcessAsync(Guid materialId, CancellationToken ct)
    {
        // 1. Load material record from DB
        // 2. Mark status = Processing
        // 3. Check content hash – skip if already vectorized
        // 4. Open PDF via IFileStorageService.OpenRead()
        // 5. Extract + chunk text with PdfProcessingService
        // 6. For each chunk: call IAIService.GetEmbeddingAsync()
        // 7. Build MaterialChunk entities with embeddings
        // 8. BulkInsert via IMaterialChunkRepository
        // 9. Mark status = Completed
        // 10. On any exception: mark status = Failed + save error message
    }
}
```

8. API Layer — Controllers

8.1 AuthController

```
[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    [HttpPost("login")]
    [AllowAnonymous]
    public async Task<IActionResult> Login([FromBody] LoginRequest req)
    // → validates email/password (BCrypt.Verify), returns JWT token

    [HttpPost("register")] // Admin only — creates Student accounts
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Register([FromBody] RegisterRequest req)
}

public record LoginRequest(string Email, string Password);
public record RegisterRequest(string Email, string FullName, string Password, UserRole
Role);
public record LoginResponse(string Token, string Role, string FullName, Guid UserId);
```

8.2 AdminController — Classes / Subjects / Chapters / Upload

```
[ApiController]
[Route("api/admin")]
[Authorize(Roles = "Admin")]
public class AdminController : ControllerBase
{
    // CLASSES
    [HttpGet("classes")] // Dapper read
    [HttpPost("classes")] // EF Core insert
    [HttpPut("classes/{id}")] // EF Core update
    [HttpDelete("classes/{id}")] // EF Core delete (cascades to subjects)

    // SUBJECTS
    [HttpGet("classes/{classId}/subjects")]
    [HttpPost("classes/{classId}/subjects")]
    [HttpPut("subjects/{id}")]
    [HttpDelete("subjects/{id}")]

    // CHAPTERS
    [HttpGet("subjects/{subjectId}/chapters")]
    [HttpPost("subjects/{subjectId}/chapters")]
    [HttpPut("chapters/{id}")]
    [HttpDelete("chapters/{id}")]

    // STUDY MATERIAL UPLOAD
    [HttpPost("upload")]
    [RequestSizeLimit(52_428_800)] // 50 MB
    public async Task<IActionResult> UploadMaterial(
        [FromForm] IFormFile file,
        [FromForm] int classId,
        [FromForm] int subjectId,
        [FromForm] int? chapterId)
    // 1. Validate file (PDF only, size check)
    // 2. Compute SHA-256 hash
    // 3. Check for duplicate hash
```

```

// 4. Save file to storage folder
// 5. Create StudyMaterial record (EF Core)
// 6. Enqueue materialId in Channel<Guid> for vectorization

[HttpGet("materials")]           // list uploaded materials with status
[HttpDelete("materials/{id}")]  // delete material + chunks + file
}

```

8.3 StudentController + ChatController

```

[ApiController]
[Route("api/student")]
[Authorize(Roles = "Student")]
public class StudentController : ControllerBase
{
    [HttpGet("classes")]           // select screen data
    [HttpGet("classes/{classId}/subjects")] // subjects for a class
    [HttpGet("subjects/{subjectId}/chapters")] // chapters for a subject
}

[ApiController]
[Route("api/chat")]
[Authorize(Roles = "Student")]
public class ChatController : ControllerBase
{
    [HttpPost("sessions")]           // create a new chat session
    [HttpGet("sessions/{sessionId}/messages")] // load history (Dapper)

    // MAIN: streaming chat endpoint
    [HttpPost("sessions/{sessionId}/messages")]
    public async Task SendMessages(
        Guid sessionId, [FromBody] SendMessagesRequest req, CancellationToken ct)
    {
        Response.Headers.Append("Content-Type", "text/event-stream");
        Response.Headers.Append("Cache-Control", "no-cache");

        // 1. Save user message to DB (EF Core)
        // 2. Embed the user's message with IAIService.GetEmbeddingAsync()
        // 3. Retrieve top-5 relevant chunks with IVectorSearchService.SearchAsync()
        // 4. Build RAG prompt (see Section 9)
        // 5. Load recent chat history from DB (Dapper, last 10 messages)
        // 6. Stream tokens from IAIService.StreamChatAsync()
        // 7. Write each token to response as SSE: "data: {token}\n\n"
        // 8. After full response, save assistant message to DB (EF Core)
        // 9. Save SourceChunkIds JSON to the assistant ChatMessage
    }
}

```

9. RAG Prompt Design

The quality of the chat experience depends entirely on the prompt templates. All three scenarios — answering questions, generating practice questions, and verifying answers — use different system prompts.

9.1 Base System Prompt (Question Answering)

```
You are an expert tutor for Class {grade}, subject "{subjectName}".
You help students understand their study material deeply and clearly.

RULES:
1. Answer ONLY based on the CONTEXT sections provided below.
2. If the context does not contain enough information, say:
   "I couldn't find this in the uploaded study material. Please ask your teacher."
3. Explain concepts clearly at a level appropriate for Class {grade} students.
4. Use examples, analogies, or simple diagrams in text when helpful.
5. Never make up facts not present in the context.

CONTEXT FROM STUDY MATERIAL:
--- CHUNK 1 (Page {page}) ---
{chunk1_text}

--- CHUNK 2 (Page {page}) ---
{chunk2_text}

... (up to 5 chunks)

Answer the student's question based on the above context.
```

9.2 Practice Question Generation Prompt

```
// Triggered when user message contains phrases like:
// "give me practice questions", "create questions", "test me", "quiz me"

ADDITIONAL INSTRUCTION (appended to base prompt):

The student has asked for practice questions.
Generate exactly 5 practice questions based on the study material context above.
Format STRICTLY as JSON:
{
  "questions": [
    {
      "id": 1,
      "question": "...",
      "type": "short-answer" | "multiple-choice",
      "options": ["A", "B", "C", "D"], // only for multiple-choice
      "correct_answer": "...",
      "explanation": "..."
    }
  ]
}

Make questions progressively harder: 2 easy, 2 medium, 1 hard.
Do NOT reveal the correct_answer or explanation in your visible response.
Only show the questions and options to the student.
```

9.3 Answer Verification Prompt

```
// Triggered after student submits an answer to a practice question

ADDITIONAL INSTRUCTION (appended to base prompt):

The student answered a practice question.
Original question: "{question_text}"
Correct answer:    "{correct_answer}"
Student's answer:  "{student_answer}"

Evaluate the student's answer:
1. State clearly: CORRECT, PARTIALLY CORRECT, or INCORRECT
2. If CORRECT: briefly praise and reinforce the concept
3. If PARTIALLY CORRECT: explain what was right and what was missing
4. If INCORRECT: gently correct with a clear explanation of the right answer
5. Reference the study material context to support your explanation
6. End with an encouraging sentence appropriate for a Class {grade} student
```

10. Frontend — React + TypeScript

10.1 Project Setup

```
# Create project
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install

# Core dependencies
npm install axios react-router-dom @tanstack/react-query
npm install tailwindcss @tailwindcss/typography
npm install react-hot-toast react-dropzone
npm install @radix-ui/react-dialog @radix-ui/react-select

# Dev dependencies
npm install -D @types/react @types/node
```

10.2 Route Structure

```
// src/App.tsx — route map

// Public routes
/login → <LoginPage />

// Admin routes (require Admin JWT role)
/admin/dashboard → <AdminDashboard />
/admin/classes → <ClassListPage />
/admin/classes/:id → <ClassDetailPage /> (subjects + chapters)
/admin/classes/:id/subjects/:sid/upload → <UploadMaterialPage />
/admin/materials → <MaterialListPage /> (with vectorization status)
/admin/users → <UserManagementPage />

// Student routes (require Student JWT role)
/student/select → <ClassSubjectSelectPage />
/student/chat/:classId/:subjectId → <ChatPage />
```

10.3 Auth Context + JWT Handling

```
// src/auth/AuthContext.tsx
interface AuthState {
  token: string | null;
  role: 'Admin' | 'Student' | null;
  userId: string | null;
  fullName: string | null;
}

// Store JWT in memory (not localStorage) for security
// Use a refresh-token cookie (httpOnly) to re-issue access tokens
// Axios interceptor adds Authorization: Bearer {token} to every request
// On 401 response: redirect to /login and clear auth state
```

10.4 Chat Page — Streaming SSE Implementation


```
// src/student/pages/ChatPage.tsx

const sendMessage = async (userText: string) => {
  // 1. Append user message to UI immediately
  setMessages(prev => [...prev, { role: 'user', content: userText }]);

  // 2. Add empty assistant message placeholder
  setMessages(prev => [...prev, { role: 'assistant', content: '' }]);

  // 3. Fetch with streaming
  const response = await fetch(`/api/chat/sessions/${sessionId}/messages`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}` },
    body: JSON.stringify({ content: userText })
  });

  const reader = response.body!.getReader();
  const decoder = new TextDecoder();

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    const text = decoder.decode(value);
    // Parse SSE lines: 'data: {token}\n\n'
    const lines = text.split('\n').filter(l => l.startsWith('data: '));
    for (const line of lines) {
      const token = line.replace('data: ', '');
      // Append to last (assistant) message
      setMessages(prev => {
        const updated = [...prev];
        updated[updated.length - 1].content += token;
        return updated;
      });
    }
  }
};
```

10.5 Key UI Components

Component	Location	Description
<ChatWindow />	student/components/ChatWindow.tsx	Message list with markdown rendering
<ChatInput />	student/components/ChatInput.tsx	Text area + send button + 'practice Qs' shortcut
<PracticeCard />	student/components/PracticeCard.tsx	Shows question, collects answer, shows verdict
<UploadZone />	admin/components/UploadZone.tsx	react-dropzone, PDF-only, shows progress
<MaterialTable />	admin/components/MaterialTable.tsx	List with vectorization status badge
<ClassTree />	admin/components/ClassTree.tsx	Collapsible: Class → Subject → Chapter

Component	Location	Description
<ProtectedRoute />	shared/components/ProtectedRoute.tsx	Checks JWT role, redirects if unauthorised
<StatusBadge />	shared/components/StatusBadge.tsx	Pending/Processing/Completed/Failed colours

11. Dependency Injection — Full Registration

```
// EduRAG.API/Extensions/ServiceRegistration.cs
public static class ServiceRegistration
{
    public static IServiceCollection AddApplication(this IServiceCollection s) =>
    {
        s // Application use cases — add as Scoped
        .AddScoped<ManageClassUseCase>()
        .AddScoped<ManageSubjectUseCase>()
        .AddScoped<ManageChapterUseCase>()
        .AddScoped<UploadMaterialUseCase>()
        .AddScoped<ChatUseCase>();

        public static IServiceCollection AddInfrastructure(
            this IServiceCollection s, IConfiguration config)
        {
            // — Database —————
            s.AddDbContext<AppDbContext>(o =>
                o.UseNpgsql(config.GetConnectionString("Default"),
                    npg => npg.UseVector())); // pgvector support

            s.AddScoped<IDbConnection>(_ =>
                new NpgsqlConnection(config.GetConnectionString("Default")));

            // — EF Core Repositories (Write) —————
            s.AddScoped<IClassRepository,      ClassRepository>()
            .AddScoped<ISubjectRepository,     SubjectRepository>()
            .AddScoped<IChapterRepository,     ChapterRepository>()
            .AddScoped<IStudyMaterialRepository, StudyMaterialRepository>()
            .AddScoped<IMaterialChunkRepository, MaterialChunkRepository>()
            .AddScoped<IChatRepository,        ChatRepository>();

            // — Dapper Queries (Read) —————
            s.AddScoped<ClassQueries>()
            .AddScoped<SubjectQueries>()
            .AddScoped<ChapterQueries>()
            .AddScoped<MaterialQueries>()
            .AddScoped<ChatQueries>();

            // — AI Services —————
            s.AddHttpClient<IAIService, OllamaAIService>(c =>
                c.BaseAddress = new Uri(config["Ollama:BaseUrl"] ??
                    "http://localhost:11434"));

            s.AddScoped<IVectorSearchService, VectorSearchService>()
            .AddScoped<PdfProcessingService>()
            .AddScoped<VectorizationProcessor>();

            // — File Storage —————
            s.AddScoped<IFileStorageService, LocalFileStorageService>();

            // — Background Job Queue —————
            s.AddSingleton(Channel.CreateUnbounded<Guid>())
            .AddHostedService<VectorizationWorker>();

            return s;
        }
    }

    public static IServiceCollection AddAuth(
        this IServiceCollection s, IConfiguration config)
    {
        s.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(o => {

```

```
        o.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(config["Jwt:Secret"]!)),
            ValidateIssuer = true, ValidIssuer = config["Jwt:Issuer"],
            ValidateAudience = true, ValidAudience = config["Jwt:Audience"],
            ClockSkew = TimeSpan.Zero
        };
    });
    s.AddAuthorization();
    return s;
}
}
```

12. Configuration Files

12.1 appsettings.json

```
{
  "ConnectionStrings": {
    "Default":
    "Host=localhost;Port=5432;Database=edurag;Username=postgres;Password=your_password"
  },
  "Jwt": {
    "Secret": "REPLACE_WITH_32+_CHAR_RANDOM_SECRET_KEY",
    "Issuer": "EduRAG",
    "Audience": "EduRAG",
    "ExpiryMinutes": 480
  },
  "Ollama": {
    "BaseUrl": "http://localhost:11434",
    "EmbedModel": "nomic-embed-text",
    "ChatModel": "llama3.2"
  },
  "Storage": {
    "BasePath": "/var/edurag/materials"
  },
  "Chunking": {
    "ChunkSize": 500,
    "ChunkOverlap": 50
  },
  "RAG": {
    "TopKChunks": 5
  }
}
```

12.2 docker-compose.yml

```
version: '3.9'
services:

  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: edurag
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: your_password
    ports:
      - '5432:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data

  ollama:
    image: ollama/ollama:latest
    ports:
      - '11434:11434'
    volumes:
      - ollama_models:/root/.ollama
    # GPU passthrough (optional, for faster inference):
    # deploy:
    #   resources:
    #     reservations:
    #       devices:
    #         - driver: nvidia
```

```
#          count: 1
#          capabilities: [gpu]

api:
  build: ./src/EduRAG.API
  ports:
    - '5000:8080'
  depends_on:
    - postgres
    - ollama
  environment:
    ConnectionStrings__Default: >
      Host=postgres;Port=5432;Database=edurag;
      Username=postgres;Password=your_password
    Ollama__BaseUrl: http://ollama:11434
  volumes:
    - materials:/var/edurag/materials

frontend:
  build: ./frontend
  ports:
    - '3000:80'
  depends_on:
    - api

volumes:
  pgdata:
  ollama_models:
  materials:
```

12.3 NuGet Packages

Package	Version	Purpose
Microsoft.EntityFrameworkCore	8.x	ORM for write operations
Npgsql.EntityFrameworkCore.PostgreSQL	8.x	PostgreSQL EF Core provider
Pgvector	0.2+	pgvector .NET support
Dapper	2.1	Micro-ORM for read queries
Npgsql	8.x	PostgreSQL ADO.NET driver
Microsoft.AspNetCore.Authentication.JwtBearer	8.x	JWT auth middleware
BCrypt.Net-Next	4.x	Password hashing
PdfPig	0.1.9	PDF text extraction
System.Threading.Channels	built-in	In-memory job queue

13. Recommended Build Order for AI Agent

Follow this order strictly. Each phase depends on the previous.

Phase 1 — Infrastructure Foundation

1. Run docker-compose up -d (Postgres + Ollama)
2. Pull Ollama models: `ollama pull nomic-embed-text && ollama pull llama3.2`
3. Create EduRAG.sln and four C# projects
4. Write all Domain entities, enums, domain events
5. Write all Application interfaces
6. Write ApplicationDbContext and EF Core configurations
7. Create and run EF Core migrations
8. Seed admin user

Phase 2 — Write Operations + Auth

9. Implement all EF Core repositories (ClassRepository, SubjectRepository, ChapterRepository, StudyMaterialRepository, MaterialChunkRepository, ChatRepository)
10. Implement JwtService (generate + validate tokens)
11. Implement AuthController (login, register)
12. Implement AdminController CRUD endpoints for Classes, Subjects, Chapters
13. Test all write + auth endpoints with Postman/curl

Phase 3 — Read Operations

14. Implement all Dapper query classes
15. Wire Dapper reads into AdminController GET endpoints
16. Implement StudentController (class/subject selection)

Phase 4 — File Upload + AI Pipeline

17. Implement LocalFileStorageService (folder creation + save)
18. Implement UploadMaterial endpoint
19. Implement PdfProcessingService (PdfPig extract + chunk)
20. Implement OllamaAIService (embed + streaming chat)
21. Implement VectorizationProcessor
22. Implement VectorizationWorker (BackgroundService + Channel)

23. Test: upload a PDF → check MaterialChunks table is populated

Phase 5 — Chat + RAG

- 24. Implement VectorSearchService (pgvector cosine query)
- 25. Implement ChatUseCase (full RAG pipeline: embed → search → prompt → stream)
- 26. Implement ChatController (create session, streaming message endpoint)
- 27. Implement ChatQueries (Dapper, load history)
- 28. Test: start chat session, ask question, verify answer is from PDF context

Phase 6 — Frontend

- 29. Set up Vite + React + TypeScript + Tailwind
- 30. Implement AuthContext, useAuth hook, ProtectedRoute
- 31. Build LoginPage (single page, shows Admin vs Student login)
- 32. Build Admin portal: Dashboard, ClassListPage, ClassDetailPage, UploadMaterialPage, MaterialListPage
- 33. Build Student portal: ClassSubjectSelectPage
- 34. Build ChatPage with streaming SSE, PracticeCard, message history

Phase 7 — Polish + Testing

- 35. Add global exception handler middleware
- 36. Add request validation (FluentValidation or DataAnnotations)
- 37. Add rate limiting on chat endpoint
- 38. Write integration tests for upload → vectorize → chat pipeline
- 39. Frontend: loading states, error handling, empty states
- 40. Dockerize API and frontend

14. Security Checklist

Area	Implementation
Password storage	BCrypt with work factor 11 (BCrypt.Net-Next)
JWT secret	Minimum 256-bit random key, stored in environment variable
JWT expiry	Access token: 8 hours. Refresh token: 7 days (httpOnly cookie)
Role enforcement	[Authorize(Roles = "Admin")] on all admin endpoints
File upload safety	Validate MIME type + extension (PDF only). Max 50 MB. Rename on disk.
SQL injection	Dapper with parameterized queries. EF Core with parameterized SQL.
CORS	Restrict to frontend origin only in production
HTTPS	Always in production. Use Let's Encrypt or self-signed for dev.
Chat scope isolation	VectorSearch always filters by classId + subjectId from session
Session ownership	Verify ChatSession.UserId == authenticated user on every message

15. Common Issues and Solutions

Issue	Solution
Ollama returns 404 on /api/embeddings	Verify model is pulled: ollama list. Pull with: ollama pull nomic-embed-text
pgvector column type error in EF migration	Ensure UseVector() is called on NpgsqlDbContextOptionsBuilder in DI registration
Embedding dimension mismatch	nomic-embed-text produces 768 dims. Ensure vector(768) in schema and EF config.
Vectorization worker not processing	Confirm Channel<Guid> is registered as Singleton and VectorizationWorker as HostedService
Chat returns empty / no context found	Check MaterialChunks table has rows. Verify ClassId + SubjectId filter matches session.
SSE stream not received in browser	Ensure Content-Type: text/event-stream header is set. Disable response buffering in IIS/nginx.
PDF text extraction returns empty	Some PDFs are image-only scans. Add Tesseract OCR fallback via Tesseract.Net.
CORS error in frontend	Add frontend origin to AllowedOrigins in CORS policy. Never use AllowAnyOrigin in production.

16. Quick Reference — All API Endpoints

Method + Path	Auth	Description
POST /api/auth/login	None	Login, returns JWT
POST /api/auth/register	Admin	Create admin or student account
GET /api/admin/classes	Admin	List all classes (Dapper)
POST /api/admin/classes	Admin	Create class (EF Core)
PUT /api/admin/classes/{id}	Admin	Update class (EF Core)
DELETE /api/admin/classes/{id}	Admin	Delete class (EF Core)
GET /api/admin/classes/{cid}/subjects	Admin	List subjects for class
POST /api/admin/classes/{cid}/subjects	Admin	Create subject
PUT /api/admin/subjects/{id}	Admin	Update subject
DELETE /api/admin/subjects/{id}	Admin	Delete subject
GET /api/admin/subjects/{sid}/chapters	Admin	List chapters
POST /api/admin/subjects/{sid}/chapters	Admin	Create chapter
PUT /api/admin/chapters/{id}	Admin	Update chapter
DELETE /api/admin/chapters/{id}	Admin	Delete chapter
POST /api/admin/upload	Admin	Upload PDF (multipart/form-data)
GET /api/admin/materials	Admin	List materials with status
DELETE /api/admin/materials/{id}	Admin	Delete material + chunks
GET /api/student/classes	Student	Classes for selection screen
GET /api/student/classes/{cid}/subjects	Student	Subjects for a class
GET /api/student/subjects/{sid}/chapters	Student	Chapters for a subject
POST /api/chat/sessions	Student	Create chat session
GET /api/chat/sessions/{sid}/messages	Student	Load chat history
POST /api/chat/sessions/{sid}/messages	Student	Send message (SSE stream)

— *End of EduRAG Technical Specification* —